tr-smoke

An AgentMPI run analysis

Abstract

This is the translation harness's smoke test: four ranks, four sections, the deterministic `function` executor in place of a model, a 60 s job budget and a 20 s barrier timeout. It completed in 0.105 s. **It is not a data point in the strong-scaling series and no speedup, efficiency or latency figure should be taken from it** — dividing the 576.74 s sequential baseline by this run's wall time gives a speedup of 5,493×, which is why `make_tables.py` selects the scaling curve by campaign label and this run carries the label `tr-smoke` rather than `real-tr`. What it is worth is two things. First, it is the end-to-end check that the harness runs: every phase executed, every contract validated, no failures, no violations, and the collectives panel is invocation-for-invocation identical to `real-tr-p4-full`'s — the same 4 scatters, 8 reduces, 8 broadcasts, 8 allreduces, 4 halo exchanges, 4 barriers, 4 gathers, the same algorithms, the same 24 messages. Second, and less obviously, it is the only run in this family whose wall clock is *entirely* AgentMPI: with no model in the loop, the 0.105 s is 114 event writes and 24 messages through the SQLite fabric, at 0.92 ms per event and a 5.5 ms median message latency under four-way write contention. Two of its three flagged warnings are artefacts of the surrogate executor rather than properties of the run, and I say which below.

1 What this run is

property	value
run	`tr-smoke`
experiment	`translation`
campaign label	`tr-smoke`
declared world size	4
ranks appearing in the log	4
executors	`function` × 4
events	114
wall time	0.11 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.01×	total busy time over wall time
parallel efficiency	0.2%	achieved parallelism per rank
idle fraction	99.8%	rank-seconds paid for and unused
peak ranks busy	1	of 4 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	4.00×	slowest rank's busy time over the mean
coordination share	76.4%	rank-seconds blocked in collectives, of those available
coordination in flight	94.3%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	12	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	24	21 eager, 3 rendezvous
tokens sent	4,128	1,558 deferred under rendezvous
tokens in / out	9,866 / 4,027	prompt and completion
cost	$0.0900	0.0223 $/kTok out

3 What the analysis notices

- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 6: the collective's own accounting disagrees with the traffic it produced.
- [note] Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- [warning] 1 collective(s) completed but recorded no duration (`halo_exchange`), so they contribute nothing to the coordination figures even though every participant blocked in them. The reported coordination share is short by exactly their blocking; it can be recovered from the event timestamps.
- [note] Collectives account for 76.4% of the run's rank-seconds, so more of the pool's time went to coordination than to work.
- [note] Too few distinct output sizes to fit β, so the reported latency parameters are medians rather than a regression.
- [note] Load imbalance is 4.00×: the slowest rank was busy far longer than the mean.
- [ok] All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

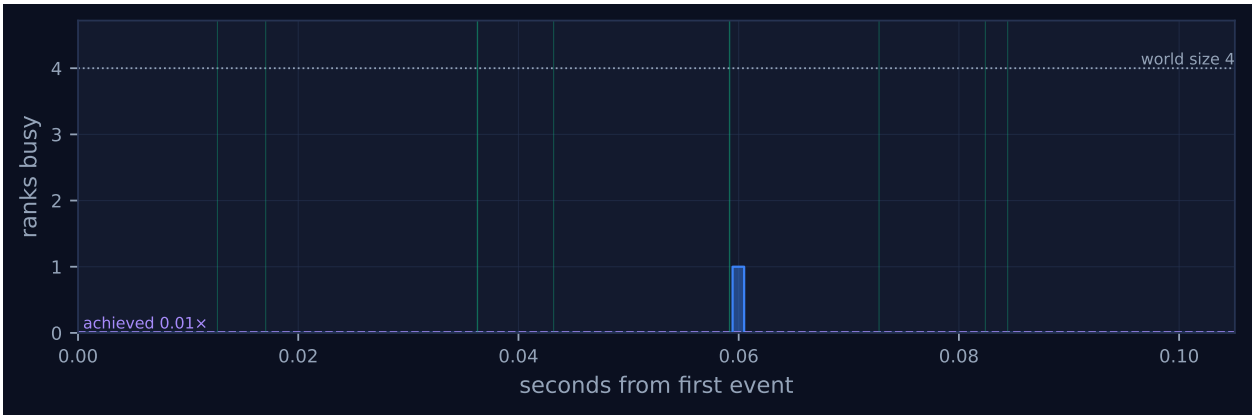

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

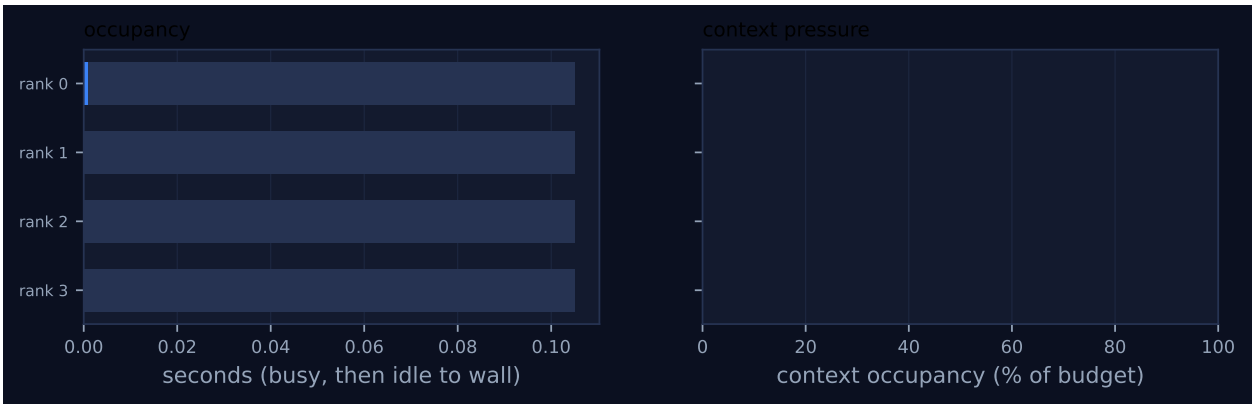

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

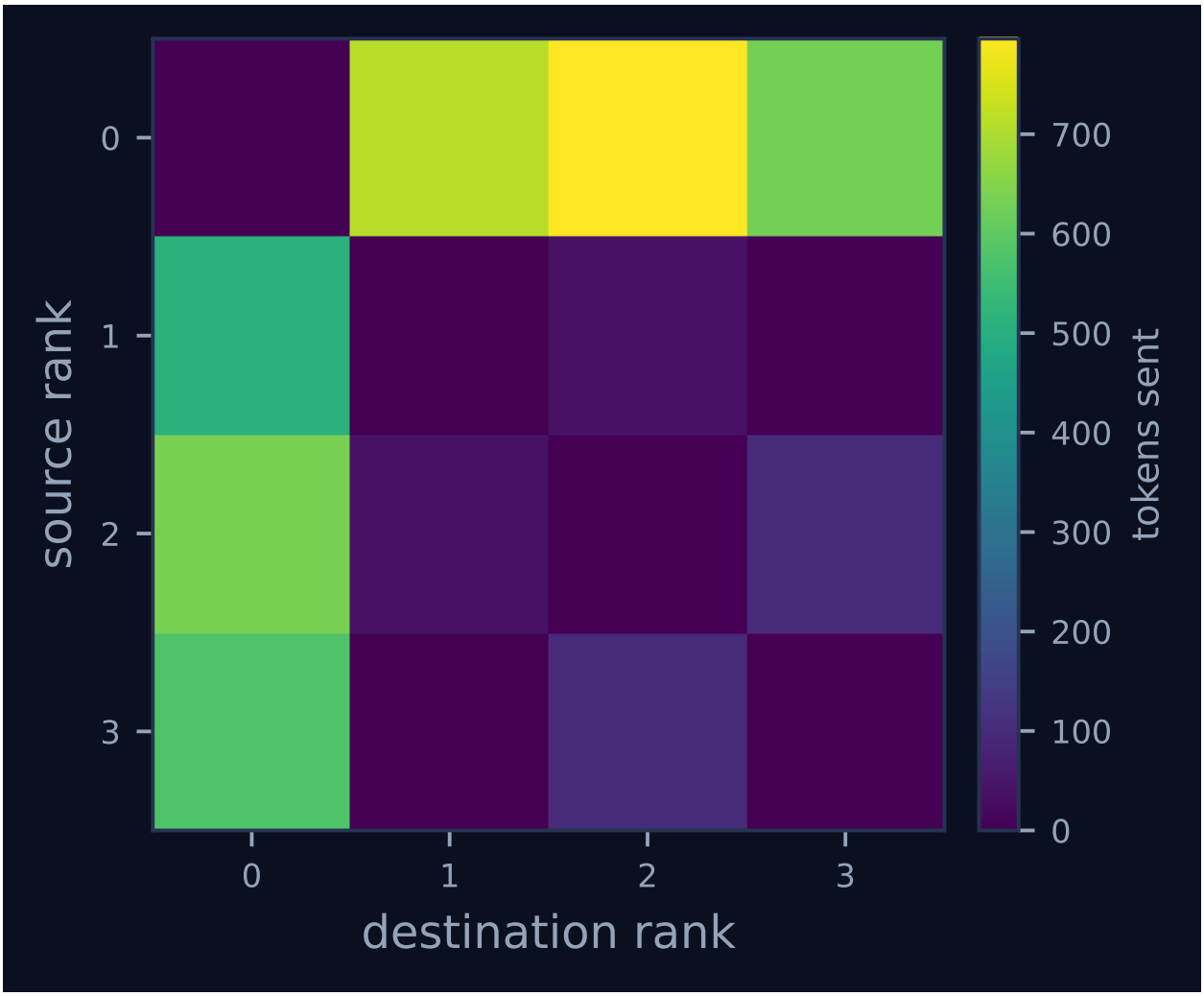

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

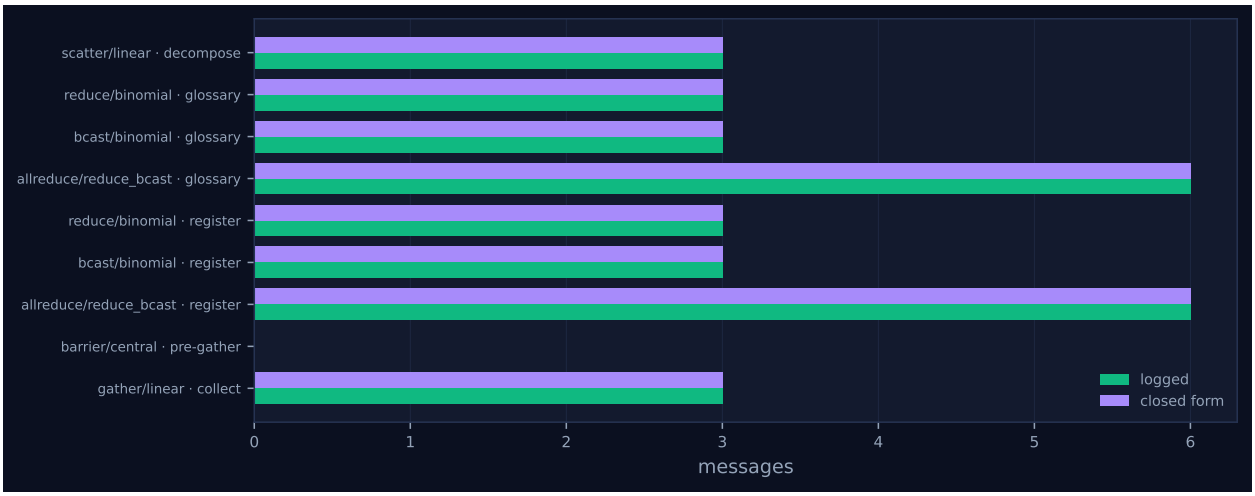

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	1.0	1	3	8	8	2,137	0.0	finalized
1	0.00	0.0	0	3	5	5	552	0.0	finalized
2	0.00	0.0	0	3	7	7	767	0.0	finalized
3	0.00	0.0	0	3	4	4	672	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	linear	decompose	4	4	1	3	3	3	1,987	0.01	0.01	✓
reduce	binomial	glossary	4	4	2	2	3	3	37	0.02	0.02	✓
bcast	binomial	glossary	4	4	2	2	3	3	141	0.02	0.01	✓
allreduce	reduce_bcast	glossary	4	4	4	4	6	6	0	0.03	0.01	✓
reduce	binomial	register	4	4	2	2	3	3	24	0.02	0.01	✓
bcast	binomial	register	4	4	2	2	3	3	24	0.02	0.01	✓
allreduce	reduce_bcast	register	4	4	4	4	6	6	0	0.03	0.01	✓
halo_exchange	?	seams	4	4	0	–	6	–	0	0.00	0.01	–
barrier	central	pre-gather	4	4	0	2	0	0	0	0.02	0.02	✓
gather	linear	collect	4	4	1	3	3	3	1,558	0.01	0.02	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has four lanes and effectively no bars, only ticks, because at this scale every interval is an instant. The viewer screenshot shows the same: four dense rows of green message ticks spread evenly across 0.10 s, with no visible blue work extent anywhere. That is the whole visual finding, and it is by construction — `synthetic_agent` returns a dictionary from a regular expression and a string join, so an "agent call" here is a function return, and eleven of the twelve report `latency_s: 0.0`.

What the ticks do show is that all four ranks ran concurrently. The first four `agent.call` events are rank 0 at 0.0131 s, rank 1 at 0.0142, rank 3 at 0.0166 and rank 2 at 0.0185, and the four lanes interleave continuously until rank 2 finalises at 0.0845 and the others at 0.102–0.105; the four terminology calls are issued 1.1, 2.4 and 1.9 ms apart. This matters because the generated findings say the opposite, and I return to it below.

The phase structure is legible in the ordering even without durations. The `scatterv` at 0.008–0.013 s sends three eager messages of 617, 741 and 629 tokens. The glossary `allreduce` occupies 0.017–0.069 s and is the most expensive phase in the run: a binomial reduce (rank 3 to rank 2, rank 1 to rank 0, rank 2 to rank 0) and a binomial broadcast back, twice over, for the glossary and then the register, 12 messages and 52 ms. Translation starts at 0.060 s, `cart_create` and the halo exchange run from 0.061 to 0.082, all four ranks issue a `revise` call, and the `pre-gather` barrier and `gather` close the run between 0.082 and 0.105.

Figure 4 shows the expected superposition of the broadcast tree and the ring, and it is more legible here than in the real runs because the payloads are uniform: every halo message is exactly 40 tokens, since the synthetic translator emits the same paragraph shape for every unit. Figure 3 has both panels near zero — occupancy because the work intervals have no extent, context because every receive is non-admitting. Figure 5 is the useful one, and all nine checkable invocations sit on the diagonal.

7 Interpretation

What the run establishes

The harness runs end to end against a four-rank world, and every stage of the pipeline that the real runs exercise was exercised here: `scatterv` of four units, four `TermSheet` calls, a `UNION` `allreduce` that produced a 5-term glossary identical at all four ranks, four `Translation` calls, a `cart_create` and `halo_exchange` on a non-periodic line, four `Revision` calls, a barrier with an empty `absent` list and a rendezvous `gather`. Zero contract violations, zero failures, zero retries, zero context rejections.

The protocol skeleton is identical to the real four-rank run. `real-tr-p4-full` logs the same ten collective invocations with the same algorithms, the same round counts, the same per-collective message counts and the same 24 messages in total. The one entry that differs is the broadcast's maximum fold depth, 2 here against 0 there, which is a property of the payloads rather than of the algorithm. The two runs otherwise differ only in payload size and in agent-call count — 12 here against 22 there, because this run takes `units = ranks = 4` where the real runs pin `-units 8`, so each rank holds a single unit and every rank issues exactly one revision. That equivalence is what makes the smoke test useful: it means the message-count and phase-structure claims in the real

runs can be regression tested deterministically, without an agent and without 300 seconds.

Because there is no model, this run also measures something the others cannot. Its 0.105 s consists of 114 event writes, 24 messages, 10 collective invocations and 8 rank lifecycle transitions, which is 0.92 ms per logged event. The 24 send-to-receive latencies have a median of 5.0 ms and a mean of 5.5 ms, with a range of 1 to 12 ms. Those figures are a useful complement to the sequential baseline's: `real-tr-p1-full` measures AgentMPI's protocol overhead at 0.38 ms per event and 0.05–0.14 ms per *degenerate* collective, with a single rank and no contention. Here four threads write the same SQLite file and a real message costs a few milliseconds. Both numbers are small enough that the conclusion is the same in either regime: the protocol is not what makes an agent run expensive.

Why this is not a measurement of anything else

Three specific hazards, all of which the campaign has already been bitten by or would be.

The wall time is 0.105 s against 308.44 s for `real-tr-p4-full`, which is the same world size and the same harness. Placed in the scaling table it would report a speedup of 5,493× and an efficiency of 1,373, and it would do so silently, because nothing in the table's arithmetic knows that one row had no model in it. `make_tables.py` avoids this by selecting on the campaign label, and the label is the only thing separating the two runs structurally.

The quality metrics are at their ceiling and mean nothing here. Coverage 1.000, consistency 1.000, `n_divergent_entities` 0, rendering verification 1.000, paragraph fidelity 1.000, length plausibility 1.000 — exactly the same six values `real-tr-p4-full` reports. What separates them is the two metrics nobody quotes: `target_script_ratio` is 0.0326 here against 0.8567 in the real run, and `residual_source_per_kchar` is 73.24 against 0.29. The synthetic "translation" is a two-character Chinese label meaning "translated paragraph" followed by the paragraph index, then the bracketed entity names, then thirty ideographic full stops — which satisfies every structural check the scorer makes — the paragraph count matches, every canonical entity is rendered, every rendering is verifiable in the text, and all four sections agree — while containing no translation at all. So six of the eight deterministic quality metrics in `common.py` are protocol checks that a placeholder passes, and two are content checks that discriminate. That is a genuinely useful result from a smoke test, and it argues for reporting `target_script_ratio` alongside consistency whenever the latter is quoted.

The cost figure is fictitious. The run is billed $0.0900 for 9,866 prompt and 4,027 completion tokens, because `cost.summarise` prices whatever token counts the fabric recorded and the synthetic executor records token counts. Any campaign-level cost total that sweeps in smoke runs is inflated by them.

The flagged findings: one real, two artefacts of the executor

[warning] *`halo_exchange/?` reports 0 messages but the fabric logged 6.* Real, and the same instrumentation gap the whole campaign has. `coll.halo_exchange` events carry only `{label, left, right}` — no `algorithm`, no `rounds`, no `messages_sent`, no `wall_s` — so the collective's self-report is empty while the fabric logged the six messages that $2(p-1)$ predicts for a non-periodic line. In the real runs the missing `wall_s` silently subtracts the halo's blocking time from `coordination_share`; here there is no blocking time to lose, so the gap shows up only as this warning and as the – in the collectives table. The sibling analysis of `real-tr-p8-full` reports the instrumentation closed by commit `ee21d79`, after these traces were taken.

[warning] *Concurrency never exceeded one rank despite a world size of 4: this ran serially.* Not true, and it is an artefact of the executor. The run did not execute serially: the event log interleaves all four ranks from 0.006 s onward, and the four `terms` calls are issued 1.1, 3.5 and 5.4 ms apart. What is true is that `total_busy_s` is 0.001. Busy time is reconstructed from `broker.claim`-to-`broker.complete` pairs, and this run's `kind_counts` contains no `broker.*` events at all — the `function` executor calls the agent inline and emits only `agent.call`, with a latency of 0.0. So achieved parallelism is computed as 0.01× and peak busy as 1 for want of any interval to measure, and the finding fires on the arithmetic rather than on the behaviour. I would call this a real false positive in `scripts/analyze_run.py` rather than a cosmetic one, because it will fire on every `function` and `simulated` run in the campaign and its wording ("this ran serially") is a claim about the run, not about the instrumentation. A guard on `executors` containing no broker would suppress it correctly.

[note] *Collectives account for 76.4% of the run's rank-seconds.* Arithmetically true and structurally meaningless in the way the sentence implies. It reads as a warning that coordination dominated the work, but with busy time at 0.001 s there is no work to dominate; the run is 0.321 collective rank-seconds out of 0.420 available because coordination is essentially all there is. The same denominator problem makes `imbalance` read exactly 1.00 — all four ranks have a busy time of zero, so they are perfectly balanced — and makes `idle_fraction` 99.8%. None of the concurrency block in the headline table should be read for this run.

[note] *Too few distinct output sizes to fit β.* Expected. Only one of the twelve invocations reports a nonzero latency (rank 0's `translate:u0`, at 1 ms), and `cost.calibrate` filters on `lat > 0`, so $n = 1$ and the branch taken is `median_only`. The resulting $\alpha = 0.001$ s and $\beta = 0.0222$ s/token describe the Python interpreter, not a model.

[ok] *All 9 checkable collectives sent exactly the number of messages their closed-form cost expression predicts.* This is the point of the run and the most valuable line in it.

No stray ranks, which is itself evidence

`stray_ranks` is empty and `n_ranks_seen` is 4 against a declared world of 4 — the only run in this family of which that is true. `real-tr-p1-full` carries seven strays, `real-tr-p2-full` eight, `real-tr-p4-full` six and `real-tr-p8-full` two. The difference is the executor: this run's ranks are in-process threads created by `launch`, so nothing external ever calls `RankRuntime.register`, whereas the real runs draw from a persistent pool of Cursor subagent processes that register themselves by index. That makes this run a control for the leak. It confirms that the spurious registrations in the real traces come from pool processes attaching to the wrong job and not from anything the harness or `create_job` does, which is consistent with the diagnosis the sibling analyses give — `register` performing an unconditional `INSERT` without consulting the `world_size` meta key — and narrows where the fix has to go.

8 Threats to this reading

The executor is a surrogate and nothing here says anything about agent behaviour. `metrics.json` reports `executors: {function: 4}`. `synthetic_agent` is forty lines of string manipulation. Every latency, every token count, every quality score and the entire cost figure are properties of that function. The only claims this document makes that survive the substitution are

about message counts, phase ordering, collective structure and fabric write cost, and I have tried to confine myself to those.

The protocol-cost numbers are one draw of a 0.1 s run and are contended in a way the real runs are not. The 0.92 ms per event and 5.5 ms median message latency come from 114 and 24 observations respectively, on a machine also running whatever else the campaign was doing, with four threads writing one SQLite file. They should be read as an order of magnitude, and specifically not as a per-message cost for a distributed deployment, where the fabric is the same file but the writers are separate processes. The comparison with `real-tr-p1-full`'s 0.38 ms per event is between an uncontended single writer and four contending ones, which is suggestive of where the difference comes from but is not an isolation of it.

The equivalence with `real-tr-p4-full` is an equivalence of skeleton, not of work. The two runs agree on collective invocations, algorithms, rounds, messages and fold depths. They do not agree on the decomposition: four units here against eight there, so one unit per rank against two, so one revision per rank against the $1, 2, 2, 1$ fan-out that dominates the real run's idle time. A regression test built on this run would not have caught that asymmetry, and the statement that the smoke test validates the protocol should be read with that boundary in mind.

The 60 s job budget and 20 s barrier timeout were never approached and so were never tested. The run finished in 0.105 s. Nothing here exercises the deadline composition that killed `real-tr-p16-full`, and a smoke test that always finishes three orders of magnitude inside its budget cannot be evidence that the timeout paths work.